Imperial College London

Department of Earth Science and Engineering

MSc in Environmental Data Science and Machine Learning

Independent Research Project

Project Plan

ClimateTech Startup-Investor Matching Tool

by

Lingxin Kong

GitHub IRP repository:

https://github.com/ese-ada-lovelace-2025/irp-lk125

Supervisors:

Maiko Schaffrath

César A. Quilodrán-Casas

Submission month and year: June 2026

Contents

Abstract **3**

1. Problem Description **3**

2. Existing System and Baseline **4**

3. Objectives **5**

4. Methodology **5**

4.1 Baseline Mapping *5*

4.2 Repeatable Outputs *5*

4.3 Rule and Semantic Scoring *6*

4.4 Evaluation with Chuan Ju *6*

5. Deliverables **6**

6. Risks and Fallback Plan **7**

7. Project Timeline **7**

References **9**

Abstract

This project improves an existing ClimateTech startup-investor matching tool rather than building a new interface from scratch. The current system already has a web form, Airtable investor data, rule scores, a gpt-4o-mini semantic layer, feedback context and a score breakdown. My role is to make the algorithm and system path easier to test, improve and explain. I will start from the existing baseline in server/services/matching.ts, record how the ranking is produced, then make controlled changes to the scoring logic and semantic matching. Chuan Ju will lead the data and evaluation side. Our work meets in the experiment loop: every algorithm change needs evidence that it makes the ranked list better, not just different.

1. Problem Description

Matching a ClimateTech startup to investors is a ranking problem. A useful result is not just a list of eligible investors. It should put the best few investors near the top, explain why they were chosen, and avoid obvious mismatches. This is close to information retrieval, where items are ranked by relevance rather than simply filtered in or out (Manning, Raghavan and Schütze, 2008).

The current matcher is useful, but it has not yet been tested like a ranking system. Rule scores can miss good investors when Airtable labels are inconsistent. The semantic layer can read richer text, but it costs time and only runs on candidates that already score well under the rules. Feedback appears in the AI prompt, which is flexible but hard to measure. My project starts from that mixed design and asks a practical question: which changes actually improve the top-ranked matches?

I will keep the existing frontend mostly unchanged. The frontend sends funding round and startup information to /api/investor-introductions/submit. The backend loads investors from Airtable, reads feedback and configuration, and then calls runMatching. The project is about improving that ranking mechanism while keeping the result format usable for the current interface.

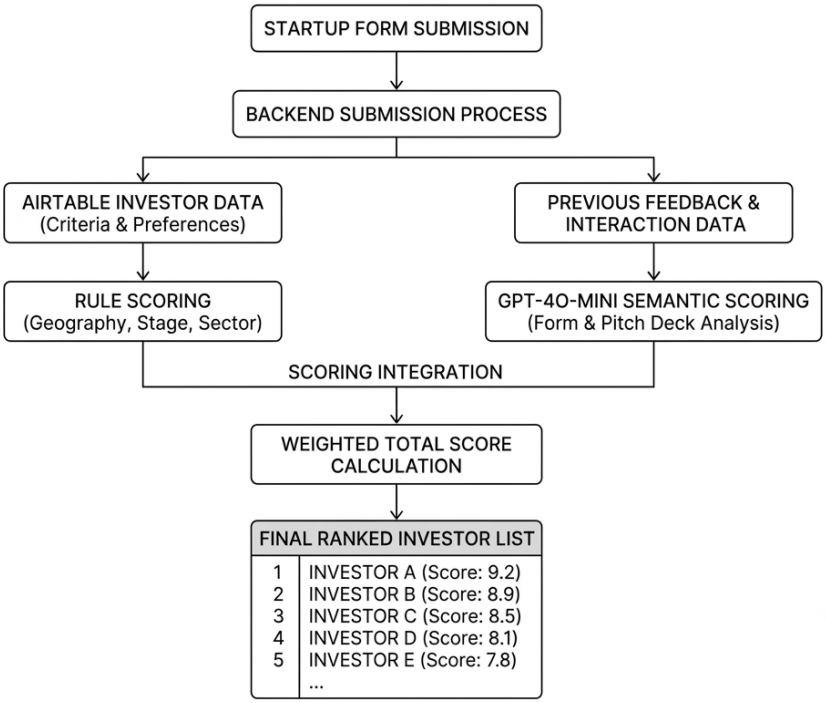


Figure 1. Current baseline path from startup form to ranked investor list.

2. Existing System and Baseline

The useful files are server/services/matching.ts, server/storage.ts, server/routes.ts, server/services/airtable.ts and client/src/pages/InvestorIntroductions.tsx. The matching.ts file contains the baseline algorithm. The storage file contains default weights, thresholds, the AI model name and the AI prompt. The routes file shows how a frontend submission becomes a matching job. The Airtable service shows which external investor fields enter the algorithm.

The baseline first computes rule scores for all investors. Stage and sector each have 20% weight, ticket size has 15%, geography has 10%, instruments have 5%, and semantic scoring has 30%. Preferred investor type changes the score through a multiplier: a match receives a small boost and a non-match receives a small penalty. The top rule-ranked candidates then go to gpt-4o-mini in batches. If the AI finds a hard exclusion, the final score is capped at 15. Candidates outside the AI batch keep semanticScore at 0, so the early rule stage has a strong effect on who gets a richer semantic check.

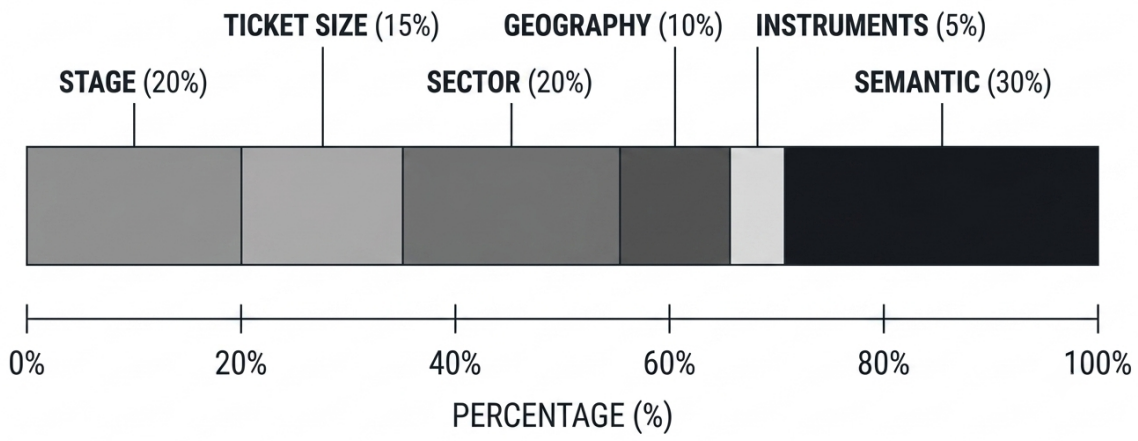


Figure 2. Baseline score weights currently used by the matching configuration.

3. Objectives

- Map the agreed baseline clearly enough that later score changes can be traced.
- Build a repeatable export or run format for totalScore, scoreBreakdown, matchReasons and aiInsight.
- Improve normalisation and rule scoring only where the evaluation work shows repeated baseline mistakes.
- Test the current gpt-4o-mini semantic route against simpler text similarity where the data allows it.
- Keep the frontend result format stable while the backend ranking logic changes.
- Produce an evidence trail that connects each algorithm change to evaluation results.

4. Methodology

4.1 Baseline Mapping

I will begin with a short map of runMatching. It will cover how fundingRound and startup objects are built, how investors are parsed from Airtable, how rule scores are calculated, how preferred investor type changes the score, how AI candidates are selected, and how final results are sorted. This is the reference point for experiments, not a one-off code reading exercise.

4.2 Repeatable Outputs

The next step is a repeatable experiment runner or export format. For each investor, I want to save input identifiers, rule scores, semantic score, total score, matchReasons and aiInsight. This lets Chuan evaluate matches without copying data from the interface. It also makes debugging less vague, because a score change can be traced to one scoring dimension.

4.3 Rule and Semantic Scoring

I will make algorithm changes in small steps. Normalisation comes first, because many bad rankings may come from labels rather than the formula itself. After that I will change rule scoring only when the error analysis supports it. For semantic matching, I will keep the current gpt-4o-mini route as one candidate approach and compare it with embedding-style text similarity if the dataset is large enough. Sentence embeddings are a reasonable comparison point because they turn text into vectors that can be compared with cosine similarity (Reimers and Gurevych, 2019).

If we later collect enough labelled examples, I will consider a light learning-to-rank experiment. That is a stretch goal, not the first move. Learning-to-rank methods are designed for ordered relevance tasks (Burges et al., 2005; Liu, 2009), but they need labels that are consistent enough to train on. I do not want to spend the project training a model on weak labels if a simpler and more explainable ranking change works better.

4.4 Evaluation with Chuan Ju

Chuan owns the evidence line of the project: data audit, label design, baseline reproduction, metrics and error analysis. My work plugs into that by producing stable outputs for comparison. We will use ranking metrics such as precision@k, mean reciprocal rank and NDCG. NDCG is useful here because it rewards highly relevant matches appearing near the top of the list (Järvelin and Kekäläinen, 2002). The final decision should not depend on one metric only; qualitative error cases matter too, especially where investor preferences are sparse.

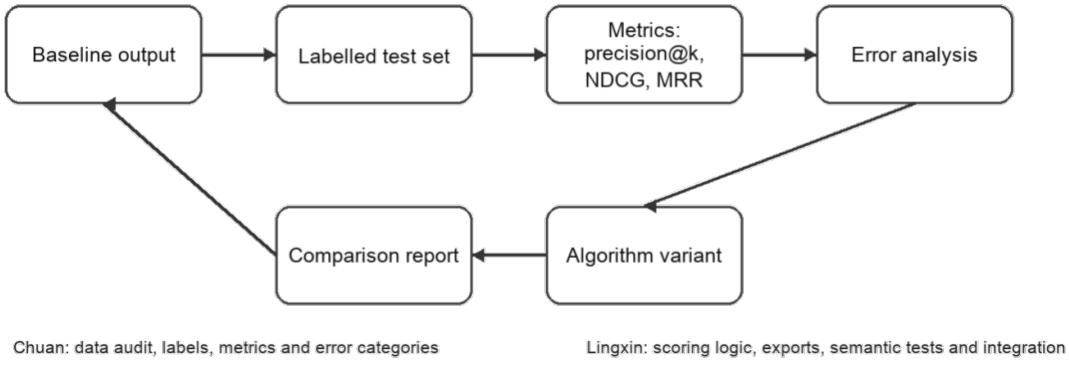


Figure 3. Shared evaluation loop between Chuan's evidence work and my algorithm work.

5. Deliverables

- A mapped baseline showing where data enters, how scores are calculated, and how ranked results are returned.

- A repeatable run or export format for baseline and variant comparisons.
- A set of controlled algorithm variants covering normalisation, rule scoring, semantic scoring and feedback use.
- A final integrated matcher that keeps the existing result format and can be explained from its score breakdown.
- Documentation that records final weights, thresholds, AI settings, limitations and suggested future work.

6. Risks and Fallback Plan

Risk	Likely effect	Fallback
Sparse or messy Airtable fields	Rule scores look worse than the formula really is.	Prioritise label normalisation and missing-data notes before changing weights.
Limited labelled matches	Supervised learning-to-rank may not be reliable.	Use rule variants, text similarity tests and careful error analysis instead.
AI scoring cost or failure	Semantic scores may be slow or inconsistent.	Keep the candidate limit, log failures, and test a cheaper text similarity baseline.
Frontend compatibility	A good algorithm change could break the working interface.	Keep output fields stable: investor, scoreBreakdown, matchReasons and aiInsight.

7. Project Timeline

15-21 Jun

- My work: Map matching.ts, routes, Airtable fields and the current scoring flow. I will focus on understanding the existing baseline before changing it.
- Chuan's parallel work: Audit Airtable fields, missing values and label consistency so we know which data problems may affect the baseline.
- Shared checkpoint: Agree the baseline definition and list the fields needed for later comparison runs.

22-28 Jun

- My work: Build a repeatable baseline run/export with score breakdown, AI insight and enough logging for later experiments.
- Chuan's parallel work: Prepare clean field rules, missing-data notes and the first version of the relevance labelling approach.

- Shared checkpoint: Run the first baseline sample and check that the exported fields are useful for evaluation.

29 Jun-5 Jul

- My work: Produce evaluation-friendly outputs and adjust the run format if Chuan's labels need extra score details or match reasons.
- Chuan's parallel work: Create the first labelled test set and define the first metrics, including precision@k, MRR and NDCG.
- Shared checkpoint: Check that labels, score explanations and ranked results refer to the same investor matches.

6-12 Jul

- My work: Fix clear normalisation problems and improve rule scoring one change at a time, starting with the weakest mismatch patterns.
- Chuan's parallel work: Review sector, stage and geography mismatch cases and record whether each change helps or hurts the same examples.
- Shared checkpoint: Compare each rule variant against the original baseline before keeping it.

13-19 Jul

- My work: Test semantic scoring with gpt-4o-mini and, if feasible, a simpler embedding-style text similarity baseline.
- Chuan's parallel work: Review text quality from startup and investor profiles, and group useful or weak feedback examples.
- Shared checkpoint: Decide which text signals and feedback signals are reliable enough to keep.

20-26 Jul

- My work: Improve how feedback affects ranking, then freeze the strongest algorithm variant and document its settings.
- Chuan's parallel work: Run metrics, ablation tables and error analysis on the strongest variants.
- Shared checkpoint: Choose the final model from evidence, explainability and workflow constraints.

27 Jul-2 Aug

- My work: Connect the final logic to the Airtable/result flow, check frontend compatibility and complete the implementation handover notes.
- Chuan's parallel work: Write final evaluation evidence, limitations and the results summary needed for the report.

- Shared checkpoint: Confirm that the core project content is complete before the checking and testing period begins.

3-9 Aug

- My work: Write the algorithm and system sections of the final report, including the baseline, variants, implementation choices and final integration notes.
- Chuan's parallel work: Write the evaluation sections, including labelled data, metrics, result tables and error analysis.
- Shared checkpoint: Combine the first full report draft and check that the evidence matches the implemented system.

10-16 Aug

- My work: Run a final project check on the matching workflow and support a short real-client experience survey by preparing the tested system and fixing practical issues.
- Chuan's parallel work: Lead the real-client experience survey, collect comments and summarise usability feedback.
- Shared checkpoint: Add survey findings, limitations and final checks to the report before polishing the submission.

References

Burges, C., Shaked, T., Renshaw, E., Lazier, A., Deeds, M., Hamilton, N. and Hullender, G. (2005) 'Learning to Rank using Gradient Descent', Proceedings of the 22nd International Conference on Machine Learning, pp. 89-96.

Järvelin, K. and Kekäläinen, J. (2002) 'Cumulated gain-based evaluation of IR techniques', ACM Transactions on Information Systems, 20(4), pp. 422-446.

Liu, T.-Y. (2009) 'Learning to Rank for Information Retrieval', Foundations and Trends in Information Retrieval, 3(3), pp. 225-331.

Manning, C.D., Raghavan, P. and Schütze, H. (2008) Introduction to Information Retrieval. Cambridge: Cambridge University Press.

Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks', Proceedings of EMNLP-IJCNLP 2019, pp. 3982-3992.